Fpt University

TRƯỜNG ĐẠI HỌC FPT

CAPSTONE PROJECT REPORT

Report 2 – Project Management Plan

II. Project Management Plan 2

1. Overview 2
    1.1 Scope & Estimation 2
    1.2 Project Objectives 4
    1.3 Project Risks 6
2. Management Approach 8
    2.1 Project Process 8
    2.2 Quality Management 11
    2.3 Training Plan 12
3. Project Deliverables 13
4. Responsibility Assignments 14
5. Project Communications 15
6. Configuration Management 17
    6.1 Document Management 17
    6.2 Source Code Management 17
    6.3 Tools & Infrastructures 18

II. Project Management Plan

1. Overview

1.1 Scope & Estimation

Product scope. SME-FraudGuard is a multi-tenant B2B SaaS platform that helps SMEs in several industries detect anomalous online payment transactions. It ingests, normalizes and evaluates transactions, raises alerts and supports investigation. Business response decisions stay with the enterprise. Each enterprise has one workspace. The Free plan detects with a Dynamic Rule Engine; the Subscription plan uses AI with rule fallback and is granted manually by the Platform Admin in v1. The scope follows the business specification v1 and registration v2; the shared 0–100 risk score, Low/Medium/High/Critical levels and the Risk Score Gateway of the 25/09 version are removed.

Actors: Platform Admin, Implementation Staff, SME Admin, Operator, Investigator, SME client system (REST/Webhook caller), AI service (system actor).

In scope:

- Accounts and tenancy – platform-provisioned accounts with five roles, backend authorization and tenant isolation; one enterprise, one workspace, one active rule set (ACC-01 to ACC-06).
- Ingestion – REST API with API key, Webhook and CSV/Excel, each with its own field mapping to the workspace's internal fields; validation, workspace-wide duplicate and conflict handling (ING-01 to ING-09).
- Rule engine – dynamic rules evaluated per transaction on standard and custom fields; one Rule Alert per transaction listing all matched rules (RUL-01 to RUL-05).
- Configuration testing – Staff test mappings and rules on samples without creating production data (TST-01 to TST-07).
- Plans – Free and Subscription entitlements enforced in the backend; Subscription granted manually by the Platform Admin (PLN-01 to PLN-07).
- AI Alert and investigation – Subscription workspaces only, testable with a clearly labeled simulated AI adapter; the real model is pending the AI contract (AIM-01 to AIM-03, CAS-01 to CAS-06).
- Supporting deliverables – demo data, containerized demo deployment and the documentation set.

Out of scope:

- Automatically blocking accounts, cancelling orders, rejecting payments or intervening in customer systems; any shared risk score, severity level, Risk Score Gateway or rule risk points (removed by BM03 and registration v2).

- Online payments, QR and automatic reconciliation (Subscription is granted manually in v1); commercial quotas on accounts, rules, transactions or storage.
- Multiple projects or configuration sets per workspace; historical-aggregation and time-window rules (v2); real AI model training and any claim that AI outperforms rules (awaiting the AI contract, OPEN-05).
- Native mobile applications, production-grade high availability, and PCI-DSS/AML compliance claims.

Construction priorities (MoSCoW). Huỳnh Khả Tú works full-time on the AI track from the start of the project, so only the other members build the rule-based flow (see 1.2). The team keeps the rule-based Free flow through the REST and CSV/Excel channels as Must. That Must scope (40 man-days) is larger than the capacity (35 man-days); the gap is stated in 1.2 and in risk 2. Webhook and the other items are Should (only if velocity exceeds the plan) or deferred. Deferred and not estimated: cases and investigation workflow, onboarding documents and Staff assignment, notifications and email, reports and export, platform health, sensitive-field encryption, the real AI model, and plan catalogue administration.

Estimation. Complexity levels are unchanged: Simple 3–4, Medium 4–6, Complex 6–10 man-days. Each item covers requirement analysis, design, coding, unit and integration testing. Estimates are proposals for the team to calibrate. Codes such as ACC, ING, RUL, TST, PLN and AC refer to rules and acceptance scenarios of the business specification.

#	WBS Item	Complexity	Est. Effort (man-days)
1	**Access & tenant isolation [Must]**		**8**
1.1	Platform-provisioned accounts and JWT login; forced password change at first login follows when capacity allows (ACC-02, ACC-03)	Simple	3
1.2	Backend role checks for five roles, workspace scope, tenant filter, cross-tenant tests (ACC-05, AC-01)	Medium	5
2	**Workspace setup [Must]**		**3**
2.1	Seeded enterprise and workspace, minimal onboarding status and handover record (ONB-01)	Simple	3
3	**Fields, mapping & ingestion (REST, CSV/Excel) [Must]**		**14**
3.1	Field definitions (type, required, sensitivity) and per-channel mapping with type conversion (ING-03 to ING-06)	Medium	5
3.2	REST ingestion: validation, workspace-wide duplicate/conflict handling (ING-06, section 6.2)	Medium	4
3.3	CSV/Excel import: header check, row-by-row processing, row-level error report (ING-09)	Medium	5
4	**Rule engine & Rule Alerts [Must]**		**11**
4.1	Rule model, draft/published versions, enable/disable (section 7)	Medium	5

4.2	Evaluation engine (type-aware operators, empty-value handling) and one Rule Alert per transaction (RUL-01, RUL-02)	Complex	6
5	**Frontend integration [Must]**		**4**
5.1	Wire the transactions, import, rules and alerts screens of the existing dashboard to the backend API	Medium	4
	Total Must		**40**
S1	[Should] Configuration test: dry run on samples, nothing persisted (TST-01 to TST-06)	Simple	3
S2	[Should] Webhook channel with HMAC over the raw body, shared ingestion pipeline	Simple	3
S3	[Should] API key issue/revoke, hashed storage, shown once (KEY-01 to KEY-04)	Medium	4
S4	[Should] Manual Free/Subscription grant and backend entitlement check (PLN-01 to PLN-07)	Medium	4
S5	[Should] Backend integration of the simulated AI adapter, rule fallback and AI Alert, labeled as simulated; the AI service and its contract belong to the AI track (AIM-02, AIM-03)	Medium	6
	Total Should (unfunded)		**20**

1.2 Project Objectives

The objective is to design, build, test and demonstrate the rule-based end-to-end flow of SME-FraudGuard within the confirmed term (project start 25/09/2026, coding 02/11/2026 to 18/12/2026, final report 20/12/2026): a workspace is set up, transactions arrive through the REST and CSV/Excel channels (Webhook as capacity allows), are mapped and validated, evaluated against active rules, and surface as Rule Alerts in the dashboard, with complete documentation and honestly reported results.

Specific objectives:

- Deliver all Must items of section 1.1 working end-to-end on the demo environment.
- Guarantee tenant isolation: no data of one workspace is accessible from another, verified by automated cross-tenant tests.
- Every alert shows its detection source and the matched rules and conditions. In v1 the source is RULE until the Should items S4 and S5 are delivered; simulated AI output is always labeled.
- No risk score or severity is calculated or displayed; no automatic blocking.

a) Quality

#	Testing Stage	Test Coverage	Notes
1	Reviewing	100% of pull requests reviewed by a member other than the author;	Three members from 06/11, so every PR has a different

		documents reviewed by one other member and the supervisor	reviewer than its author
2	Unit Test	Business logic of own code; coverage target set once coverage tooling is configured	JUnit 5 (backend); Node test runner (dashboard)
3	Integration Test	Public APIs of the delivered ingestion channels (REST, CSV/Excel) and the rule/alert path, against PostgreSQL via Testcontainers	Run in CI where Docker is available
4	System Test	Scenarios AC-01, AC-05 (REST and CSV), AC-06 to AC-11 of the business specification for the Must scope; AC-12 with S1 if delivered; Report 5 catalogue	Report 5 designed cases stay Not Run until executed
5	Acceptance Test	Main rule-based flow demonstrated to the supervisor	No open Critical/High defects at final release

b) Milestone Timeliness (%): ≥ 90% of milestones (reports and releases in section 3) delivered on or before the planned date; no milestone delayed by more than 1 week.

c) Allocated Effort (man-days): capacity is computed for the construction window 02/11/2026 – 18/12/2026. Huỳnh Khả Tú works full-time on the AI track, so the builders of the rule-based flow are Lê Hồng Đức, Nguyễn Minh Huy and, until 05/11, Nguyễn Thị Diệu Vi.

02/11 – 05/11: 4 working days × 3 builders = 12 person-days. 06/11 – 18/12: 31 working days × 2 builders = 62 person-days. Total 74 person-working-days. At the 25/09 plan's productivity of 2.8 man-days per person-week (0.56 per working day) this gives about 41 man-days; the figure is kept from the 25/09 plan and rechecked against real hours at the Sprint 1 review on 13/11/2026. About 6 man-days (15%) go to project management and documents, leaving about 35 man-days for the WBS. The AI track has about 20 man-days (35 working days × 0.56), not counted here. The previous total of 247 man-days used 22 working weeks, which did not match the 15-week term.

The Must scope is 40 man-days, so it exceeds the 35 available by 5 man-days (14%), and every Should item is unfunded. The estimates are not reduced to hide the gap. Concern: the team has not yet decided how to close it. The options are extra capacity from the AI track after the AI approach note, or reducing the Must scope in the order given in risk 2. The

decision is taken at the Sprint 1 checkpoint on 13/11/2026. Effort spent before 02/11 (Reports 1–3, training, registration v2) is taken from the tracking sheet and is not re-estimated here.

#	Activity	Effort (man-days)	Percentage
1	Requirement clarification	2	5%
2	Design	4	10%
3	Coding (frontend, backend, rule engine, DevOps)	20	49%
4	Testing (unit, integration, system, acceptance)	9	22%
5	Project Management (planning, tracking, meetings, reporting)	3	7%
6	Documentation & Other (user manual, deployment guide, final report)	3	7%
	Total	**41**	**100%**

1.3 Project Risks

Probabilities are the team's initial judgment, not measured data, and are adjusted at the sprint retrospectives. The list is reviewed at every sprint retrospective and updated in the project tracking sheet.

#	Risk Description	Impact	Possibility	Response Plans
1	The team drops from 4 to 3 members on 06/11/2026. Only two members (one backend, one frontend) then build the rule-based flow, because the third works full-time on the AI track. Database design, test data and the Scrum Master role leave with the departing member.	High	High (planned)	Phase 0 (schema, contracts, API contract) finished, reviewed and merged by 05/11; written handover note; each module understood by at least two of the remaining members; Scrum Master role moves to Nguyễn Minh Huy, who receives the Jira board and tracking sheet.
2	Compressed construction window (about 6.7 weeks) and capacity (≈ 35 man-days for the WBS) 5 man-days (14%) below the Must scope (40), with the Should items unfunded.	High	High (70%)	Checkpoint at the Sprint 1 review on 13/11 against real hours. Decide at that checkpoint how to close the gap: extra capacity from the AI track after the AI approach note, or reduce the Must scope in this order: wiring of the import screen, frontend

				integration to the rules and alerts screens only, rule versioning to a single version, then change control with the supervisor.
3	The frontend member must also write Java backend code (the rule engine) because there is one backend developer after 05/11.	Medium	High (60%)	Training already planned in 2.3; the backend leader reviews all backend PRs; the first backend PR of this member is paired with the departing member before 06/11 if possible.
4	Specification items are still proposed or open (OPEN-01 to OPEN-11), so mapping, rule and schema decisions may change after coding starts.	Medium	Medium (50%)	Build only confirmed behavior; record decisions in the business specification before implementing dependent code; keep migrations additive.
5	The AI contract, dataset and algorithm are undecided (OPEN-05); the AI track has been owned by Huỳnh Khả Tú since the start of the project. Time he spends on AI is not available for the rule-based Must scope.	Medium	High (60%)	v1 uses a labeled simulated adapter, as the specification allows; the real model is out of v1; never present simulated output as real; the AI report is time-boxed and is cut or shrunk before any Must item slips.
6	Tenant data leakage or access-control flaws: tenant filtering at the repository layer and real authentication are not implemented yet (development stub only).	High	Medium (40%)	Access group (1.1, 1.2) is the first Must item; automated cross-tenant tests; security checklist in code review; use synthetic data only until done.
7	Scope creep.	Medium	Medium (50%)	MoSCoW list in 1.1; baseline after supervisor approval; new items go through change control.

8	Integration problems between frontend and backend; the dashboard currently runs on demo data and browser storage.	Medium	Medium (40%)	API contract agreed in Phase 0 before any frontend wiring; integration week 30/11 to 06/12.
9	Learning curve of Spring Security/JWT, Flyway migrations and Testcontainers.	Medium	Medium (50%)	Training plan 2.3; project skeleton and architecture tests already in place.
10	Member unavailability. With three members, the absence of one blocks a third of the capacity.	High	Medium (30%)	Code review and documentation so two members know each module; leader reassigns tasks; the Must scope is reduced through change control, since no Should item is funded.

2. Management Approach

The project is managed with an agile approach based on Scrum, adapted for a small student team: two 2-week sprints, then an integration week and a stabilization period, planned backward from fixed milestone dates. Scope is managed through a prioritized product backlog (MoSCoW) derived from the business specification and the SRS; progress is tracked in Jira with the effort tracking sheet; quality is ensured through reviews and multi-level testing; and the supervisor is involved in weekly meetings and sprint reviews. Capstone milestones (reports 1–7, final defense) are treated as fixed dates, and sprint content is planned backward from them.

2.1 Project Process

The team applies an iterative and incremental development process based on Scrum, as illustrated in the figure below.

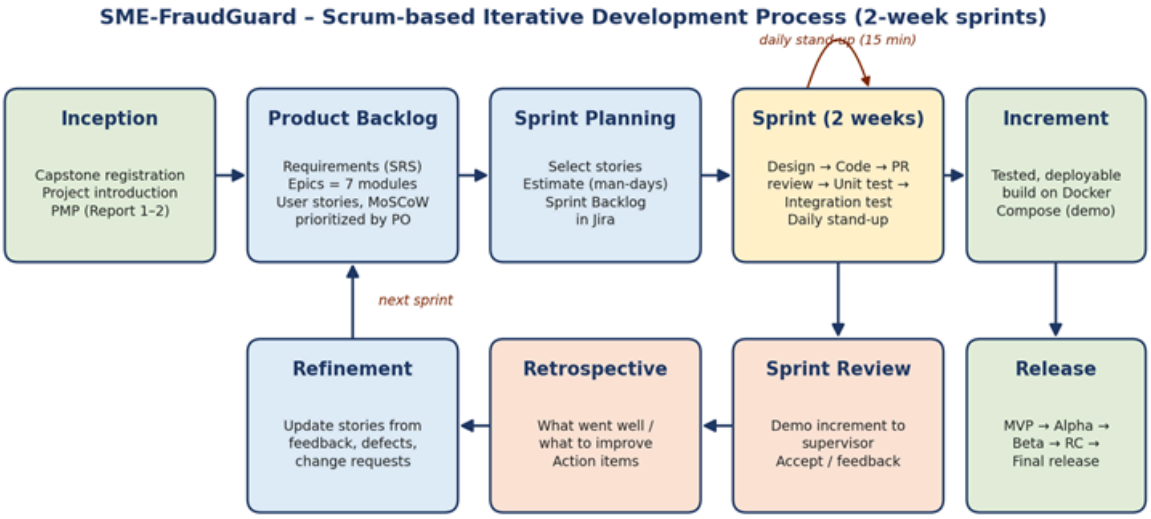


Figure 1. Scrum-based development process of SME-FraudGuard

Roles:

- **Product Owner:** Lê Hồng Đức (team leader), who represents SME and platform needs, owns the product backlog and agrees priorities with the supervisor.
- **Scrum Master:** Nguyễn Thị Diệu Vi until 05/11/2026, then Nguyễn Minh Huy (from 06/11/2026), who facilitates ceremonies, maintains Jira and the tracking sheet, and removes blockers.
- **Development Team:** Đức and Huy build the rule-based flow (Vi until 05/11/2026); Huỳnh Khả Tú works full-time on the AI track from the start of the project.
- **Supervisor:** Mr. Huỳnh Công Việt Ngữ, who reviews increments and documents and gives feedback in sprint reviews and weekly meetings.

Process phases:

- **Inception and requirements (25/09 – 01/11/2026):** team formation, topic registration, training, Reports 1–3, registration v2 and the business specification. Project skeleton and CI are prepared in parallel.
- **Construction (02/11 – 27/11/2026):** Sprint 1 (02/11 – 13/11) and Sprint 2 (16/11 – 27/11). The team has 4 members until 05/11/2026 and 3 from 06/11/2026; Phase 0 (schema, contracts) is merged by 05/11.
- **Integration (30/11 – 06/12/2026):** end-to-end run on synthetic data, tenant isolation tests, demo environment and MVP freeze; Release 1 on 06/12.
- **Stabilization & Transition (07/12 – 20/12/2026):** system, security and performance checks, execution of test cases, user manual, deployment guide, final release (18/12), final report (20/12) and defense preparation.

Detailed Timeline – Weeks 1–13 (current: Week 6). Weeks are counted from the project start on 25/09/2026 and run Friday to Thursday. The table below breaks the plan into individual weeks so progress can be tracked against the WBS in section 1.1 and the deliverables in section 3.

We	Dates	Phase / Sprint	Focus	Milestone

ek				
1	25/09 – 01/10/2026	Inception	Team formation, topic idea, initial research	–
2	02/10 – 08/10/2026	Inception	Topic registration; training (Spring Boot, Next.js, Git, Scrum/Jira); Report 1 draft	Report 1 – Project Introduction (04/10)
3	09/10 – 15/10/2026	Inception	Training and planning; Project Management Plan	Report 2 – Project Management Plan (11/10)
4	16/10 – 22/10/2026	Requirements	Business specification, registration v2, SRS sections, database and architecture drafts	–
5	23/10 – 29/10/2026	Requirements	SRS sections, project skeleton, CI pipeline	–
6	30/10 – 05/11/2026	Sprint 1 (starts 02/11)	4 members (3 build, 1 AI). Phase 0: schema migration, cross-module contracts, API contract, dependencies; handover by the member leaving	Report 3 – SRS (01/11); Sprint 1 Planning (02/11)
7	06/11 – 12/11/2026	Sprint 1	3 members from 06/11 (2 build, 1 AI). JWT login and role checks; field definitions and REST mapping; rule model	–
8	13/11 – 19/11/2026	Sprint 1 ends 13/11; Sprint 2 starts 16/11	Sprint 1 review and checkpoint (13/11) against real hours; CSV/Excel import; rule evaluation	Sprint 1 Review & Retro; Sprint 2 Planning
9	20/11 – 26/11/2026	Sprint 2	Rule Alerts; frontend wiring of transactions, import, rules and alerts	Report 4 – Software Design Document (22/11)
10	27/11 – 03/12/2026	Sprint 2 ends 27/11; Integration starts 30/11	End-to-end run on synthetic data; tenant isolation tests; demo environment	AI approach note (29/11); Sprint 2 Review & Retro

11	04/12 – 10/12/2026	Integration ends 06/12	MVP freeze; bug fixing	Release 1 – rule-based MVP (06/12)
12	11/12 – 17/12/2026	Stabilization	System, security and performance checks; execute test cases; user manual and deployment guide	Report 5 – Test Plan & Test Cases, Report 6 – User Manual & Deployment Guide (13/12)
13	18/12 – 24/12/2026	Stabilization	Final bug fixing; package Docker deployment; consolidate the final report; defense rehearsal	Final Release (18/12); Report 7 – Final Capstone Report (20/12)

Week 1 begins on the project start date, 25/09/2026. Coding starts on Monday 02/11/2026 (Week 6). The team has 4 members until 05/11/2026 and 3 from 06/11/2026 (Week 7). Huỳnh Khả Tú works on the AI track throughout.

Sprint goals. *Sprint 1 (02/11 – 13/11): Phase 0 merged by 05/11; JWT login and role checks; field definitions and REST ingestion with mapping; rule model and publish/enable/disable.*

Sprint 2 (16/11 – 27/11): CSV/Excel import; evaluation engine and Rule Alerts; dashboard wiring of transactions, import, rules and alerts. Should items start only if the 13/11 checkpoint shows spare capacity.

Integration (30/11 – 06/12): acceptance scenarios AC-01 and AC-05 to AC-11; cross-tenant tests; deployment to the demo environment; freeze.

Stabilization (07/12 – 18/12): system test execution, documents, final release.

Ceremonies: Sprint Planning (first day of the sprint), Daily Stand-up (15 minutes, online), Sprint Review with the supervisor and Sprint Retrospective (last day of the sprint), and Backlog Refinement (mid-sprint). Definition of Done: code reviewed by someone other than the author and merged, tests passing, behavior matches the cited business specification rule, feature runs on the demo environment, and related documents updated.

2.2 Quality Management

To reach the quality objectives in section 1.2, the team applies the following quality activities throughout the project:

- **Defect Prevention:** coding conventions and the static analysis the repository already provides (ESLint with the Next.js rules, TypeScript checks, ArchUnit module-boundary tests); additional Java style tooling only if the team agrees; a clear API contract from Phase 0; root-cause analysis of important defects in retrospectives.

- **Reviewing:** every document (SRS, SDD, test plan, reports) is reviewed by at least one other member and by the supervisor before baselining; every pull request needs one approval from a member other than the author and a passing CI run (backend clean verify; dashboard npm run check and build; landing page typecheck and build) before the team leader merges. Review checklists include tenant isolation and security items.
- **Unit Testing:** developers write unit tests for their own code (JUnit 5 for the Spring Boot backend, the Node test runner for the dashboard); a coverage target is set once coverage tooling is configured; tests run automatically in GitHub Actions.
- **Integration Testing:** API tests of the delivered ingestion channels and the rule/alert path with Spring Boot Test and Testcontainers (PostgreSQL), using unique workspace identifiers per test, including cross-tenant access tests; executed at the end of every sprint.
- **System Testing:** the remaining members execute the Report 5 test cases for the Must scope. All cases start as Not Run; Pass or Fail is recorded only from real execution evidence. Defects are logged in Jira with severity and priority and tracked until closed.
- **Acceptance Testing:** the main rule-based flow is demonstrated to the supervisor in sprint reviews and at the final release; the release is accepted only when no Critical/High defects remain open.

2.3 Training Plan

Several technologies in the project are new to some members. The following training is planned at the beginning of the project (mostly self-study with online courses and official documentation, followed by internal knowledge-sharing sessions).

Training Area	Participants	When, Duration	Waiver Criteria
Java 21, Spring Boot 3 (Spring Web, Spring Security + JWT, Spring Data JPA), Flyway	Đức, and Huy who will write backend code (risk 3)	Weeks 4–7	Mandatory; the 25/09 version said Java 17, the repository uses 21
Next.js, React, TypeScript, CSS Modules	Frontend members	Done / ongoing	The existing dashboard uses CSS Modules (not Tailwind)
PostgreSQL & multi-tenant database design, Testcontainers	All members	Weeks 4–6	Handover by the database owner before 06/11
Docker, Docker Compose, GitHub Actions CI	All members	Weeks 4–6	Waived for members who already use them

Git, GitHub (pull request review), Scrum & Jira	All members	Week 1	Done
API security basics: OWASP API Top 10, JWT, RBAC, API key handling	All members	Week 6	Mandatory

Python, FastAPI and machine-learning training stays with the AI owner (Tú) only; it is removed from the mandatory list for the other members because the real AI model is out of v1.

3. Project Deliverables

The main internal and external deliverables of the project are listed below. The project term is 25/09/2026 – 20/12/2026; all due dates fit inside this window and will only be adjusted to match the official schedule announced by the department. The AI work (owner: Huỳnh Khả Tú) runs on a dedicated full-time track in parallel with the rule-based construction.

#	Deliverable	Due Date	Notes
1	Capstone Project Register form (FA26SE215)	14/09/2026	Registered topic and team
2	Report 1 – Project Introduction	04/10/2026	Context, existing solutions, proposed solution, scope
3	Report 2 – Project Management Plan	11/10/2026; revised 04/11/2026	This document (rebaselined)
4	Report 3 – Software Requirement Specification (SRS)	01/11/2026	Sections agreed so far; functional requirements follow the finalized section 3
5	Report 4 – Software Design Document (architecture, database, API specification, AI design)	22/11/2026	Architecture and database design diagrams; reviewed by supervisor
6	AI approach note (owner: Huỳnh Khả Tú, full-time AI track, about 20 man-days) – replaces the AI Evaluation Report	29/11/2026	Describes the Normal/Anomaly contract and the simulated adapter; experiments as far as AI track capacity allows, results never fabricated. Pending supervisor approval, to be requested at the next weekly

			meeting and before 29/11. If not approved, the report stays but is limited to one baseline experiment on a public dataset, with its limitations stated.
7	Release 1 – rule-based MVP: access and tenant isolation, workspace setup, REST and CSV/Excel ingestion, Dynamic Rule Engine with Rule Alerts, dashboard	06/12/2026	End of integration week; Should items only if delivered
8	Report 5 – Test Plan & Test Cases (executed results)	13/12/2026	Unit, integration, system and acceptance test results; designed cases remain Not Run until executed
9	Report 6 – User Manual & Deployment Guide	13/12/2026	For Platform Admin, SME Admin and Operator
10	Final Release – source code, Docker deployment package, demo data	18/12/2026	GitHub tag v1.0.0
11	Report 7 – Final Capstone Report & presentation slides	20/12/2026	Consolidated documentation
12	Final Defense	TBD (week of 20/12/2026)	According to the Capstone Council schedule

4. Responsibility Assignments

The main responsibilities of the team members are assigned as follows (Lê Hồng Đức – Project Leader, Backend, Database, Documentation; Huỳnh Khả Tú – AI track, Tester, Documentation; Nguyễn Minh Huy – Frontend, Rule engine backend from 06/11, Scrum Master from 06/11, Tester, Documentation; Nguyễn Thị Diệu Vi – Database, Tester, Documentation, until 05/11/2026). This matrix is a proposal until the team agrees: Đức, Tú and Huy confirm it before 05/11/2026, including who reviews backend pull requests.

D~Do; R~Review; S~Support; I~Informed; <blank>- Omitted

Responsibility	**DucLHSE182106**	**TuHKSE182134**	**HuyNMSE181524**	**ViNTDSE181505**

Project Planning & Tracking (Scrum Master from 06/11: HuyNM)	D	R	D	S
Prepare Report 1 and Report 2 (including this revision)	D	R	R	S
Prepare SRS (Overview, Business Rules, NFRs)	D	R	R	S
Prepare SRS (Use Cases, Screen Flows, Functional Requirements)	R	D	D	S
Architecture Design & API Contract	D	R	R	S
Database Design and Phase 0 migration (handover to DucLH by 05/11)	R	I	I	D
Backend – access & tenant isolation (group 1)	D	I	R	S
Backend – workspace setup, fields, mappings, ingestion (groups 2, 3)	D	I	R	S
Backend – rule engine & Rule Alerts (group 4), from 06/11	R	I	D	S
Frontend – transactions, import, rules, alerts screens (group 5)	R	I	D	I
Demo data & test fixtures	S	I	D	S
AI Evaluation Report / AI approach note (due 29/11); AI owner since project start	R	D	I	S
Report 4 – SDD	D	R	R	S
Report 5 – Test Plan & Test Cases	R	D	D	S
Unit Testing (own code)	D	D	D	D
Integration & System Testing, defect tracking	D	D	D	S
Deployment (demo environment, CI)	D	I	I	I
Report 6 – User Manual & Deployment Guide	R	D	D	I
Report 7 – Final Report & Presentation	D	S	S	I

Huy takes on the rule-engine backend work in Java from 06/11 (risk 3), which is why the backend leader reviews all backend pull requests. Tú is not assigned to the rule-based Must scope; he joins Report 5 test execution after the AI approach note (29/11). Handover by 05/11/2026 (member leaving): merged Phase 0 (migration, cross-module contracts, API contract, dependencies); database design notes; test data and fixtures; tracking sheet and Jira board (to Nguyễn Minh Huy as the new Scrum Master); a short note of open decisions and known gaps.

5. Project Communications

The communication activities of the project are planned as follows:

Communication Item	Who/ Target	Purpose	When, Frequency	Type, Tool, Method(s)

Daily Stand-up	All team members	Report what was done, what is planned and blockers	Daily, 15 minutes (21:00)	Online meeting – Discord / Google Meet
Sprint Planning	Team (supervisor informed)	Select backlog items, estimate, assign tasks for the sprint	First day of each sprint (bi-weekly), 1–2 hours	Offline at campus or Google Meet; Jira sprint board
Sprint Review	Team, Supervisor	Demo the increment, collect feedback, accept stories	Last day of each sprint (bi-weekly), 1 hour	Offline / Google Meet; live demo on Docker environment
Sprint Retrospective	Team	Identify improvements, review risks and action items	Last day of each sprint (bi-weekly), 30 minutes	Google Meet; notes on Google Drive
Supervisor Meeting	Supervisor (Mr. Huỳnh Công Việt Ngữ), Team	Report progress, discuss requirements/documents, get guidance	Weekly, 1 hour	Offline at FPT University or Google Meet; meeting minutes on Google Drive
Milestone Report Submission	Supervisor, Capstone Council	Submit reports 1–7 and product releases	According to milestone dates (section 3)	E-mail / FPT learning system; Google Drive
Task & Defect Tracking	Team	Assign and follow tasks, defects and change requests	Continuously	Jira (issues, burndown); GitHub Issues linked to PRs
Code Review	Developers	Review pull requests to ensure code quality and security	Every pull request, within 24 hours	GitHub Pull Requests
Urgent Issues	Team, Team Leader	Resolve blockers and urgent problems quickly	As needed	Zalo / Discord group chat, phone call

Handover session	Leaving member and remaining members	Walk through Phase 0, database design, fixtures, tracking sheet	By 05/11/2026	Google Meet; note stored in 01_Management
Sprint 1 checkpoint	Team and supervisor	Compare delivered Must items (40 man-days) against the 35 man-day capacity and real hours; decide how to close the capacity gap; reduce the Must scope through change control if behind	13/11/2026	Sprint Review
Contract change notice	Developers on the MVP tracks	Announce any change to a cross-module api package or shared file before merging	As needed	Group chat and PR description

6. Configuration Management

6.1 Document Management

All project documents are stored in a shared Google Drive folder owned by the team leader and shared with all members and the supervisor.

- **Folder structure:** 01_Management (plans, meeting minutes, tracking sheet), 02_Requirements, 03_Design, 04_Test, 05_Reports (Report 1–7), 06_AI_Experiments, 07_Release, 99_Archive.
- **Naming convention:** FA26SE215_<DocumentName>_v<major>.<minor> (e.g. FA26SE215_Report3_SRS_v1.2.docx). Minor version is increased for each reviewed change; major version is increased when a document is baselined/submitted.
- **Change tracking:** every document has a Record of Changes table (date, A/M/D, person in charge, description); large changes are made with Track Changes/comments and approved by the document owner.
- **Review & baseline:** a document is baselined only after internal review and supervisor review; submitted versions are copied to 05_Reports and never edited again (new changes create a new version).
- **Technical documents close to the code** (API specification via OpenAPI/Swagger, README, deployment guide, architecture decision records) are version-controlled in the Git repository under /docs.

6.2 Source Code Management

Source code is managed with Git on GitHub in a private organization repository (monorepo: /frontend, /backend, /ai-service, /infra, /database, /docs), shared with the supervisor.

- **Branching:** main holds the stable, reviewed state. Work happens on short-lived branches named feat/, fix/ or docs/ followed by the task name, and the team

leader reviews and merges them into main. This replaces the Git flow model (develop, feature/<JIRA-ID>-…, release, hotfix) of the 25/09 version, which is too heavy for three members and about six weeks of coding. main is protected; releases are Git tags on main.

- **Pull requests:** main is protected; merging requires a pull request with at least 1 approval from a member other than the author, passing CI (lint, tests, build) and a linked task. Keep pull requests to about 400 changed lines and rebase on main before opening.
- **Commit convention:** Conventional Commits (feat:, fix:, docs:, test:, refactor:, chore:) to keep a readable history and generate changelogs.
- **Versioning & releases:** Semantic Versioning with Git tags – v0.1.0 (MVP), v0.2.0 (Alpha), v0.3.0 (Beta), v0.9.0 (Release Candidate), v1.0.0 (Final). Each release has release notes and a Docker image build.
- **Configuration & secrets:** environment settings in .env files that are not committed (.env.example is committed); secrets stored in GitHub Actions Secrets; Spring profiles (dev, test, demo) for configuration; database changes managed with Flyway migrations.
- **AI artifacts:** training code and notebooks in /ai-service; datasets are not committed (download scripts only); experiments and model versions tracked with MLflow, and the model version is returned in every AI response.

6.3 Tools & Infrastructures

Category	Tools / Infrastructure
Technology	Frontend: Next.js (App Router), React, TypeScript, CSS Modules, Recharts, exceljs. Backend: Java 21, Spring Boot 3.3, Spring Data JPA, Spring Security (to be added), Flyway, ArchUnit. Database: PostgreSQL 16. AI service: generic service behind the aiclient contract; framework not selected.
Hosting	AWS hosting for the web portals and backend, Cloudflare DNS, Neon-hosted PostgreSQL (Report 4 architecture); exact AWS product not selected.
Testing	JUnit 5, Spring Boot Test, Testcontainers, ArchUnit, Node test runner, Postman (optional)
Version Control	GitHub (source code), Google Drive (documents)
CI/CD & Deployment	GitHub Actions (backend clean verify, dashboard npm run check and build, landing page typecheck and build); Docker and Docker Compose for local and demo environments
Diagramming & Documentation	draw.io, PlantUML, Figma, Google Docs/Sheets/Slides, MS Office
Project management	Jira (backlog, sprints, tasks, defects), Google Sheets (schedule, effort tracking)
Communication	Discord, Zalo, Google Meet, E-mail